Causal Forest and Its Variants in `grf`: Estimation, Inference, Visualization & R Applications

Mingyu Shu
shumingyu@foxmail.com

May 3, 2026

Outline

Causal Forest Fundamentals

Causal Forest Fundamentals

The CATE Problem

Goal

Estimate the Conditional Average Treatment Effect: $\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]$

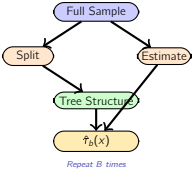
- ATE: $\mathbb{E}[Y(1) - Y(0)]$ — who benefits *on average*
- CATE: $\tau(x)$ — who benefits *most* given covariates
- Under unconfoundedness: $(Y(1), Y(0)) \perp W \mid X$
- Fundamental problem: never observe both potential outcomes

Causal Forest: Algorithm

Key design:

1. **Honest splitting**: separate samples for structure and estimation
2. **Adaptive weighting**: forest-based kernel $\alpha_i(x)$
3. **Ensemble**: average over B causal trees

$\hat{\tau}(x) = \sum_{i=1}^{n} \alpha_i(x)\, Y_i$



grf::causal_forest()

```
library(grf)
cf <- causal_forest(X, Y, W,
  num.trees = 2000, sample.fraction = 0.5,
  honesty = TRUE, honesty.fraction = 0.5, seed = 42)
tau_hat <- predict(cf, X)$predictions
```

- `honesty = TRUE`: honest splitting (default, recommended)
- `honesty.fraction`: proportion reserved for estimation
- Always set `seed` for reproducibility

RATE: Assessing Heterogeneity

RATE: Definition

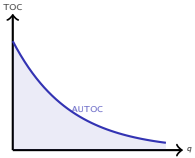
Rank-Weighted Average Treatment Effect

Yadlowsky et al. (2021): RATE measures how well CATE estimates **prioritize** treatment. RATE $= \int_0^1 \mathrm{TOC}(q)\, dq$

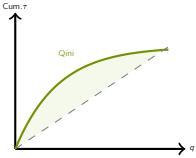
- If $\tau(x)$ is constant $\Rightarrow$ RATE $= 0$ (no heterogeneity)
- RATE > 0 with significant p-value $\Rightarrow$ heterogeneity detected
- Two variants: **AUTOC** and **Qini**

TOC and Qini Curves

TOC Curve



Qini Curve



grf::rank_average_treatment_effect()

```
rate <- rank_average_treatment_effect(cf, cf$predictions, target = "AUTOC")
print(rate)   # estimate, std.err, p-value
plot(rate)    # TOC curve
```

- `target = "AUTOC"`: area under TOC (default)
- `target = "Qini"`: Qini curve variant
- Low p-value $\Rightarrow$ significant heterogeneity

Best Practice

Use train/test split to avoid overfitting: fit CF on train, evaluate RATE on test.

Multi-Arm Causal Forest

`grf::multi_arm_causal_forest()`

Problem

When $K > 2$ treatment arms, estimate: $\tau_k(x) = \mathbb{E}[Y(k) - Y(0) \mid X = x], \quad k = 1, \ldots, K-1$

- Based on multivariate R-learner (Nie & Wager, 2021)
- Treatment W is a factor with K levels (baseline = first level)
- Joint estimation of all $\tau_k(x)$ via GRF kernel weights
- Supports `get_scores()` for doubly robust inference

Multi-Arm CF: R Code

```
cf_multi <- multi_arm_causal_forest(X, Y, W, num.trees = 2000)
pred <- predict(cf_multi, X)  # array: dims = (n, K-1)
tau_1 <- pred$predictions[,1]  # CATE for arm 1 vs control
tau_2 <- pred$predictions[,2]  # CATE for arm 2 vs control
ate <- average_treatment_effect(cf_multi, target = "A")
```

Visualization:

```
boxplot(pred$predictions, names = paste0("Arm",1:(K-1)),
  col = c("steelblue","coral","forestgreen"), ylab = "CATE")
```

Instrumental Forest

`grf::instrumental_forest()`

IV Forest

Estimate LATE with instrumental variables: $\tau_{IV}(x) = \frac{\mathbb{E}[Y|Z=1,X=x]-\mathbb{E}[Y|Z=0,X=x]}{\mathbb{E}[W|Z=1,X=x]-\mathbb{E}[W|Z=0,X=x]}$

- Handles **non-compliance** and **endogeneity**
- Part of GRF framework (Athey et al., 2019)
- Used in OHIE analysis (Hollenbach et al., 2024)
- Supports `get_scores()` for doubly robust LATE

IV Forest: R Code

```
ivf <- instrumental_forest(X, Y, W, Z, num.trees = 2000)
tau_iv <- predict(ivf, X)$predictions
late <- average_treatment_effect(ivf)
dr_scores_iv <- get_scores(ivf)
```

- Z: instrument (e.g., lottery assignment)
- W: actual treatment received
- Y: outcome
- Returns conditional LATE estimates

Causal Survival Forest

grf::causal_survival_forest()

Causal Survival Forest

Cui, Kosorok, Sverdrup & Wager (2023). Estimating HTE with right-censored data via causal survival forests. *JRSS-B*, 85(2).

- Estimates CATE with **right-censored** outcomes
- Two targets: **RMST** or **survival probability**
- RMST: $\tau(x) = \mathbb{E}[\min(T(1), h) - \min(T(0), h) \mid X = x]$
- Survival prob.: $\tau(x) = P[T(1) > h \mid X = x] - P[T(0) > h \mid X = x]$
- Also supports **continuous treatment** W

Causal Survival Forest: R Code

```
csf <- causal_survival_forest(X, Y, W, D,
  target = "RMST", horizon = 10, num.trees = 2000)
# D: event indicator (0=censored, 1=event)
# Y: observed time, W: treatment, horizon: follow-up time
tau_surv <- predict(csf, X)$predictions
ate_surv <- average_treatment_effect(csf, target = "A")
```

Key arguments:

- `target = "RMST"`: restricted mean survival time (default)
- `target = "survival.probability"`: absolute risk difference
- `horizon`: required, defines the estimand

LM Forest (RDD Application)

`grf::lm_forest()`

Linear Model Forest

Estimates the conditionally linear model: $Y = c(x) + h_1(x)W_1 + \cdots + h_K(x)W_K \quad \text{at } X = x$

- **Generalizes** causal forest and multi-arm causal forest
- Can estimate **heterogeneous RDD effects** by setting $W_1 = \mathbb{1}\{Z \geq c\}$ and $W_2 = Z$
- Added in grf v2.2.0 (2022)
- Official grf documentation includes RDD example

LM Forest for RDD: R Code

```
# Regression Discontinuity with lm_forest
# W1 = treatment indicator, W2 = running variable
n <- 2000; Z <- runif(n, -4, 4); cutoff <- 0
W1 <- as.numeric(Z >= cutoff)
W2 <- Z
W_mat <- cbind(W1, W2)
lmf <- lm_forest(X, Y, W_mat, num.trees = 2000)
pred <- predict(lmf)
# pred$predictions[,1]: heterogeneous RDD effect
tau_rdd <- pred$predictions[,1]
```

Ref: grf documentation — lm_forest reference

Quantile Forest

`grf::quantile_forest()`

Quantile Forest

Estimate conditional quantiles: $Q_\alpha(x) = \inf\{y : F(y \mid X = x) \geq \alpha\}$

- Part of GRF framework (Athey et al., 2019)
- Useful for **distributional treatment effects**
- Compare quantiles of $Y(1)$ vs $Y(0)$ at different α

Quantile Forest: R Code

```
qf <- quantile_forest(X, Y, quantiles = c(0.1, 0.5, 0.9),
  num.trees = 2000)
q_pred <- predict(qf, X)
# Distributional TE: compare quantiles by treatment group
q_t <- quantile_forest(X[W==1,], Y[W==1],
  quantiles = seq(0.1, 0.9, 0.1))
q_c <- quantile_forest(X[W==0,], Y[W==0],
  quantiles = seq(0.1, 0.9, 0.1))
```

Probability Forest

grf::probability_forest()

Probability Forest

Estimate conditional class probabilities: $P(Y = k \mid X = x), \quad k = 1, \ldots, K$

- Non-parametric **multiclass classification** via forests
- Useful for estimating **propensity scores** $e(x) = P(W = 1 \mid X = x)$
- Can be used as nuisance estimator for causal forests
- Returns calibrated probability estimates

Probability Forest: R Code

```
# Estimate propensity scores
pf <- probability_forest(X, as.factor(W), num.trees = 2000)
e_hat <- predict(pf, X)$predictions[,2]  # P(W=1|X)
# Use as input to causal forest
cf <- causal_forest(X, Y, W, W.hat = e_hat, num.trees = 2000)
# Or: multi-class classification
pf_multi <- probability_forest(X, Y_factor, num.trees = 2000)
prob <- predict(pf_multi, X)$predictions
```

Survival Forest

`grf::survival_forest()`

Survival Forest

Estimate conditional survival function $S(t \mid X = x) = P(T > t \mid X = x)$ with right-censored data.

- **Non-causal**: estimates survival curves, not treatment effects
- Useful as nuisance estimator for `causal_survival_forest()`
- Supports `prediction.times` for specific time points
- Based on log-rank splitting criterion

Survival Forest: R Code

```
sf <- survival_forest(X, Y, D,
  prediction.times = c(1, 3, 5), num.trees = 2000)
# D: event indicator (0=censored, 1=event)
# Y: observed time
pred <- predict(sf, X)
# pred$predictions: survival curves at specified times
```

Regression Forest Variants

grf::regression_forest()

Regression Forest

Estimate conditional mean $\mu(x) = \mathbb{E}[Y \mid X = x]$.

- Standard GRF regression (not causal)
- Used internally by causal forests for nuisance estimation
- Supports variance estimation and confidence intervals

```
rf <- regression_forest(X, Y, num.trees = 2000)
mu_hat <- predict(rf, X)$predictions
```

grf::ll_regression_forest()

Local Linear Regression Forest

Combines random forest with local linear corrections for improved boundary behavior.

- Better performance at **boundaries** of feature space
- Reduces bias from standard forest splitting
- Useful when smoothness of $\mu(x)$ is expected

```
llf <- ll_regression_forest(X, Y, num.trees = 2000,
  enable.ll.splitting = TRUE)
mu_ll <- predict(llf, X)$predictions
```

grf::boosted_regression_forest()

Boosted Regression Forest

Applies gradient boosting to regression forest for improved prediction.

- Combines forest ensemble with **boosting**
- Tuning: `boosting.iterations`, `boosting.step.size`

```
brf <- boosted_regression_forest(X, Y, num.trees = 2000,
  boosting.iterations = 5)
mu_boost <- predict(brf, X)$predictions
```

grf::multi_regression_forest()

Multi-Task Regression Forest

Jointly estimate multiple outcomes $\mu_k(x) = \mathbb{E}[Y_k \mid X = x]$ for $k = 1, \ldots, K$.

- Shares forest structure across multiple outcomes
- Borrows strength when outcomes are correlated
- Y is a matrix with K columns

```
mrf <- multi_regression_forest(X, Y_matrix, num.trees = 2000)
pred <- predict(mrf, X)  # array: (n, K)
```

Analysis Tools in grf

grf::test_calibration()

```
blp <- test_calibration(cf)
print(blp)
# p-value < 0.05 => significant heterogeneity detected
```

Interpretation

- BLP test: can we predict $\tau(x)$ better than constant ATE?
- Significant p-value $\Rightarrow$ heterogeneity exists
- Also provides variance decomposition

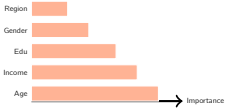
grf::best_linear_projection()

```
blp <- best_linear_projection(cf, A = X)
print(blp)
# Projects CATE onto linear functions of A
# Shows which covariates drive heterogeneity
```

- Projects $\hat{\tau}(x)$ onto linear basis of covariates
- Identifies which variables explain heterogeneity
- Supports `target.sample = "overlap"` for overlap weights

grf::variable_importance()

```
vi <- variable_importance(cf)
barplot(sort(vi, decreasing = TRUE), horiz = TRUE,
  col = "coral", border = NA, xlab = "Importance", las = 1)
```



grf::average_treatment_effect()

```
# Doubly robust ATE
ate <- average_treatment_effect(cf, target = "A")
# ATE on the treated (ATT)
att <- average_treatment_effect(cf, target = "ATT")
# Overlap-weighted ATE
ate_overlap <- average_treatment_effect(cf, target = "A",
  subset = cf$predictions > quantile(cf$predictions, 0.25) &
           cf$predictions < quantile(cf$predictions, 0.75))
```

- Uses doubly robust scores for valid inference
- `target = "A"`: ATE, `target = "ATT"`: ATT
- Supports `subset` argument for subgroup ATE

grf::get_scores() and grf::split_frequencies()

```
# Doubly robust scores (for policytree)
dr <- get_scores(cf)         # causal forest
dr_m <- get_scores(cf_multi) # multi-arm CF
dr_s <- get_scores(csf)      # causal survival CF
dr_iv <- get_scores(ivf)     # instrumental forest

# Split frequencies: which variables are used most?
sf <- split_frequencies(cf, max.depth = 4)
barplot(colSums(sf), las = 2, col = "steelblue")
```

grf::tune_forest()

```
# Cross-validated tuning of forest hyperparameters
tuned_cf <- tune_forest(cf,
  tune.parameters = c("min.node.size", "sample.fraction",
    "mtry", "alpha", "imbalance.penalty"))
# Or: tune during fitting
cf_tuned <- causal_forest(X, Y, W,
  tune.parameters = "all", num.trees = 2000)
```

- Tunes: `min.node.size`, `sample.fraction`, `mtry`, `alpha`, `imbalance.penalty`
- Uses out-of-bag error for selection
- `tune.parameters = "all"`: tune all parameters

Policy Learning with policytree

`policytree::policy_tree()`

Policy Learning

Learn a treatment rule $\pi : \mathcal{X} \to \{1, \ldots, K\}$ maximizing welfare: $W(\pi) = \mathbb{E}\left[\sum_{k=1}^{K} \mathbb{1}(\pi(X_i) = k) \cdot Y_i(k)\right]$

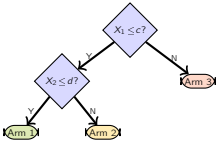
- Athey & Wager (2021, *Econometrica*): doubly robust
- Zhou et al. (2023, *OR*): multi-action extension
- Policy = **shallow decision tree** for interpretability
- R package: `policytree` (companion to `grf`)

Policy Tree: Algorithm

Two-step procedure:

1. Estimate **doubly robust scores** Γ_i using `grf::get_scores()`
2. Find **globally optimal** depth-k tree via `policytree`

$$\hat{\pi} = \arg\max_{\pi \in \Pi_k} \sum_{i=1}^{n} \Gamma_{i,\pi(X_i)}$$



Policy Tree: R Code

```
library(policytree)
# Step 1: Doubly robust scores from multi-arm CF
dr_scores <- get_scores(cf_multi)
# Step 2: Learn optimal policy tree
policy <- policy_tree(X, dr_scores, depth = 2)
assigned <- predict(policy, X)
print(policy)  # show tree structure
```

Complete grf Function Summary

All grf Forest Functions

Function	Purpose	Key Output
`causal_forest`	Binary CATE	$\hat{\tau}(x)$
`multi_arm_causal_forest`	K-arm CATE	$\hat{\tau}_k(x)$
`causal_survival_forest`	CATE w/ censoring	RMST / survival diff
`instrumental_forest`	LATE with IV	$\hat{\tau}_{IV}(x)$
`lm_forest`	Linear model / RDD	$\hat{h}_k(x)$
`quantile_forest`	Conditional quantiles	$Q_\alpha(x)$
`probability_forest`	Class probabilities	$P(Y=k \mid X)$
`survival_forest`	Survival curves	$S(t \mid X)$
`regression_forest`	Conditional mean	$\hat{\mu}(x)$
`ll_regression_forest`	Local linear mean	$\hat{\mu}(x)$
`boosted_regression_forest`	Boosted mean	$\hat{\mu}(x)$
`multi_regression_forest`	Multi-task mean	$\hat{\mu}_k(x)$

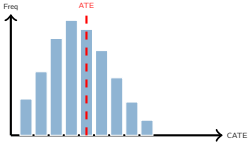
All grf Analysis Functions

Function	Purpose
`average_treatment_effect`	Doubly robust ATE / ATT
`rank_average_treatment_effect`	RATE (AUTOC / Qini)
`best_linear_projection`	BLP of CATE on covariates
`get_scores`	Doubly robust scores
`test_calibration`	Omnibus calibration test
`variable_importance`	Feature importance
`split_frequencies`	Variable split counts
`get_forest_weights`	Forest kernel weights
`get_tree`	Extract single tree
`merge_forests`	Merge forest objects
`tune_forest`	Cross-validated tuning

Visualization Guide

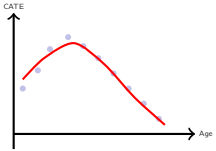
Viz 1: CATE Distribution

```
hist(tau_hat, breaks = 30, col = "steelblue",
  border = "white", xlab = "CATE", main = "CATE Distribution")
abline(v = mean(tau_hat), col = "red", lwd = 2)
```



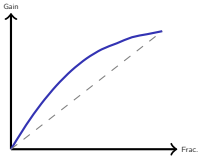
Viz 2: CATE vs. Covariate

```
plot(X[,1], tau_hat, pch = 16, cex = 0.5,
  col = rgb(0.2,0.4,0.8,0.3), xlab = "Age", ylab = "CATE")
lines(lowess(X[,1], tau_hat), col = "red", lwd = 2)
```



Viz 3: Qini / TOC Curve

```
rate_qini <- rank_average_treatment_effect(
  cf, cf$predictions, target = "Qini")
plot(rate_qini, main = "Qini Curve",
  xlab = "Fraction treated", ylab = "Cumulative gain")
```

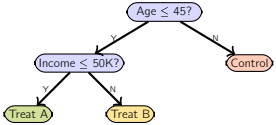


Viz 4: 2D CATE Heatmap

```
library(ggplot2)
grid <- expand.grid(
  x1 = seq(min(X[,1]),max(X[,1]),length=50),
  x2 = seq(min(X[,2]),max(X[,2]),length=50))
grid$tau <- predict(cf, as.matrix(grid))$predictions
ggplot(grid, aes(x1,x2,fill=tau)) + geom_tile() +
  scale_fill_viridis_c() + labs(x="Age",y="Income",fill="CATE")
```

Viz 5: Policy Tree Diagram

```
plot(policy)  # built-in plot from policytree
```



Viz 6: Survival Curves

```
csf <- causal_survival_forest(X, Y, W, D,
  target = "survival.probability", horizon = 5)
tau_surv <- predict(csf, X)$predictions
plot(X[,1], tau_surv, pch = 16, cex = 0.5,
  col = rgb(0.8,0.2,0.2,0.3), xlab = "Age",
  ylab = "Survival difference")
```

Complete Workflow

End-to-End Workflow

Key Packages

- `grf`: All forest variants — grf-labs.github.io/grf
- `policytree`: Optimal policy trees — grf-labs.github.io/policytree

Thank You!

Questions & Discussion

shumingyu@foxmail.com